PEP 612: Parameter Specification Variables

Status: Final · Created: 18-Dec-2019 · Public domain / CC0

Abstract

There currently are two ways to specify the type of a callable, the Callable[[int, str], bool] syntax defined in 484, and callback protocols from PEP 544. Neither of these support forwarding the parameter types of one callable over to another callable, making it difficult to annotate function decorators. This PEP proposes typing.ParamSpec and typing.Concatenate to support expressing these kinds of relationships.

Motivation

The existing standards for annotating higher order functions don’t give us the tools to annotate the following common decorator pattern satisfactorily:

add_logging\ , a decorator which logs before each entry into the decorated function, is an instance of the Python idiom of one function passing all arguments given to it over to another function. This is done through the combination of the *args and **kwargs features in both parameters and in arguments. When one defines a function (like inner\ ) that takes (*args, **kwargs) and goes on to call another function with (*args, **kwargs), the wrapping function can only be safely called in all of the ways that the wrapped function could be safely called. To type this decorator, we’d like to be able to place a dependency between the parameters of the callable f and the parameters of the returned function. 484 supports dependencies between single types, as in def append(l: typing.List[T], e: T) -> typing.List[T]: ...\ , but there is no existing way to do so with a complicated entity like the parameters of a function.

Due to the limitations of the status quo, the add_logging example will type check but will fail at runtime. inner will pass the string “B” into takes_int_str\, which will try to add 7 to it, triggering a type error. This was not caught by the type checker because the decorated takes_int_str was given the type Callable[..., Awaitable[int]] (an ellipsis in place of parameter types is specified to mean that we do no validation on arguments).

Without the ability to define dependencies between the parameters of different callable types, there is no way, at present, to make add_logging compatible with all functions, while still preserving the enforcement of the parameters of the decorated function.

With the addition of the ParamSpec variables proposed by this PEP, we can rewrite the previous example in a way that keeps the flexibility of the decorator and the parameter enforcement of the decorated function.

Another common decorator pattern that has previously been impossible to type is the practice of adding or removing arguments from the decorated function. For example:

With the addition of the Concatenate operator from this PEP, we can even type this more complex decorator.

Specification

ParamSpec Variables

Declaration

A parameter specification variable is defined in a similar manner to how a normal type variable is defined with typing.TypeVar.

The runtime should accept bound\ s and covariant and contravariant arguments in the declaration just as typing.TypeVar does, but for now we will defer the standardization of the semantics of those options to a later PEP.

Valid use locations

Previously only a list of parameter arguments ([A, B, C]) or an ellipsis (signifying "undefined parameters") were acceptable as the first "argument" to typing.Callable . We now augment that with two new options: a parameter specification variable (Callable[P, int]\ ) or a concatenation on a parameter specification variable (Callable[Concatenate[int, P], int]\ ).

where parameter_specification_variable is a typing.ParamSpec variable, declared in the manner as defined above, and concatenate is typing.Concatenate.

As before, parameters_expression\ s by themselves are not acceptable in places where a type is expected

User-Defined Generic Classes

Just as defining a class as inheriting from Generic[T] makes a class generic for a single parameter (when T is a TypeVar\ ), defining a class as inheriting from Generic[P] makes a class generic on parameters_expression\ s (when P is a ParamSpec).

By the rules defined above, spelling a concrete instance of a class generic with respect to only a single ParamSpec would require unsightly double brackets. For aesthetic purposes we allow these to be omitted.

Semantics

The inference rules for the return type of a function invocation whose signature contains a ParamSpec variable are analogous to those around evaluating ones with TypeVar\ s.

Just as with traditional TypeVars\ , a user may include the same ParamSpec multiple times in the arguments of the same function, to indicate a dependency between multiple arguments. In these cases a type checker may choose to solve to a common behavioral supertype (i.e. a set of parameters for which all of the valid calls are valid in both of the subtypes), but is not obligated to do so.

The constructors of user-defined classes generic on ParamSpec\ s should be evaluated in the same way.

The semantics of Concatenate[X, Y, P] are that it represents the parameters represented by P with two positional-only parameters prepended. This means that we can use it to represent higher order functions that add, remove or transform a finite number of parameters of a callable.

This also means that while any function that returns an R can satisfy typing.Callable[P, R], only functions that can be called positionally in their first position with a X can satisfy typing.Callable[Concatenate[X, P], R].

There are still some classes of decorators still not supported with these features:

* those that add/remove/change a **variable** number of parameters (for example, functools.partial will remain untypable even after this PEP) * those that add/remove/change keyword-only parameters (See Concatenating Keyword Parameters_ for more details).

The components of a ParamSpec

A ParamSpec captures both positional and keyword accessible parameters, but there unfortunately is no object in the runtime that captures both of these together. Instead, we are forced to separate them into *args and **kwargs\ , respectively. This means we need to be able to split apart a single ParamSpec into these two components, and then bring them back together into a call. To do this,

we introduce P.args to represent the tuple of positional arguments in a given call and P.kwargs to represent the corresponding Mapping of keywords to values.

Valid use locations

These "properties" can only be used as the annotated types for *args and **kwargs\ , accessed from a ParamSpec already in scope.

Furthermore, because the default kind of parameter in Python (\ (x: int)\ ) may be addressed both positionally and through its name, two valid invocations of a (*args: P.args, **kwargs: P.kwargs) function may give different partitions of the same set of parameters. Therefore, we need to make sure that these special types are only brought into the world together, and are used together, so that our usage is valid for all possible partitions.

Semantics

With those requirements met, we can now take advantage of the unique properties afforded to us by this set up:

* Inside the function, args has the type P.args\ , not Tuple[P.args, ...] as would be with a normal annotation (and likewise with the **kwargs\ )

```
        * This special case is necessary to encapsulate the heterogeneous contents
          of the args/kwargs of a given call, which cannot be expressed
          by an indefinite tuple/dictionary type.
```

* A function of type Callable[P, R] can be called with (*args, **kwargs) if and only if args has the type P.args and kwargs has the type P.kwargs\ , and that those types both originated from the same function declaration. * A function declared as def inner(*args: P.args, **kwargs: P.kwargs) -> X has type Callable[P, X].

With these three properties, we now have the ability to fully type check parameter preserving decorators.

To extend this to include Concatenate, we declare the following properties:

* A function of type Callable[Concatenate[A, B, P], R] can only be called with (a, b, *args, **kwargs) when args and kwargs are the respective components of P, a is of type A and b is of type B. * A function declared as def inner(a: A, b: B, *args: P.args, **kwargs: P.kwargs) -> R has type Callable[Concatenate[A, B, P], R]. Placing keyword-only parameters between the *args and **kwargs is forbidden.

Note that the names of the parameters preceding the ParamSpec components are not mentioned in the resulting Concatenate. This means that these parameters can not be addressed via a named argument:

This is not an implementation convenience, but a soundness requirement. If we were to allow that second calling style, then the following snippet would be problematic.

Inside of bar, we would get TypeError: foo() got multiple values for argument 'x'. Requiring these concatenated arguments to be addressed positionally avoids this kind of problem, and simplifies the syntax for spelling these types. Note that this also why we have to reject signatures of the form (*args: P.args, s: str, **kwargs: P.kwargs) (See Concatenating Keyword Parameters_ for more details).

If one of these prepended positional parameters contains a free ParamSpec\ , we consider that variable in scope for the purposes of extracting the components of that ParamSpec. That allows us to spell things like this:

The type of twice in the above example is Callable[Concatenate[Callable[P, int], P], int], where P is bound by the outer Callable. This has the following semantics:

Backwards Compatibility

The only changes necessary to existing features in typing is allowing these ParamSpec and Concatenate objects to be the first parameter to Callable and to be a parameter to Generic. Currently Callable expects a list of types there and Generic expects single types, so they are currently mutually exclusive. Otherwise, existing code that doesn't reference the new interfaces will be unaffected.

Reference Implementation

The Pyre type checker supports all of the behavior described above. A reference implementation of the runtime components needed for those uses is provided in the pyre_extensions module. A reference implementation for CPython can be found here.

Rejected Alternatives

Using List Variadics and Map Variadics

We considered just trying to make something like this with a callback protocol which was parameterized on a list-type variadic, and a map-type variadic like so:

However, there are some problems with trying to come up with a consistent solution for those type variables for a given callable. This problem comes up with even the simplest of callables:

Any time where a type can implement a protocol in more than one way that aren't mutually compatible, we can run into situations where we lose information. If we were to make a decorator using this protocol, we would have to pick one calling convention to prefer.

The core problem here is that, by default, parameters in Python can either be called positionally or as a keyword argument. This means we really have three categories (positional-only, positional-or-keyword, keyword-only) we're trying to jam into two categories. This is the same problem that we briefly mentioned when discussing .args and .kwargs. Fundamentally, in order to capture two categories when there are some things that can be in either category, we need a higher level primitive (\ ParamSpec\ ) to capture all three, and then split them out afterward.

Defining ParametersOf

Another proposal we considered was defining ParametersOf and ReturnType operators which would operate on a domain of a newly defined Function type. Function would be callable with, and only with ParametersOf[F]. ParametersOf and ReturnType would only operate on type variables with precisely this bound. The combination of these three features could express everything that we can express with ParamSpecs.

We decided to go with ParamSpec\ s over this approach for several reasons:

* The footprint of this change would be larger, as we would need two new operators, and a new type, while ParamSpec just introduces a new variable. * Python typing has so far has avoided supporting operators, whether user-defined or built-in, in favor of destructuring. Accordingly, ParamSpec based signatures look much more like existing Python. * The lack of user-defined operators makes common patterns hard to spell. unwrapping is odd to read because F is not actually referring to any callable. It's just being used as a container for the parameters we wish to propagate. It would read better if we could define an operator RemoveList[List[X]] = X and then unwrapping could take F and return Callable[ParametersOf[F], RemoveList[ReturnType[F]]]. Without that, we unfortunately get into a situation where we have to use a Function-variable as an improvised ParamSpec, in that we never actually bind the return type.

In summary, between these two equivalently powerful syntaxes, ParamSpec fits much more naturally into the status quo.

Concatenating Keyword Parameters

In principle the idea of concatenation as a means to modify a finite number of positional parameters could be expanded to include keyword parameters.

However, the key distinction is that while prepending positional-only parameters to a valid callable type always yields another valid callable type, the same cannot be said for adding keyword-only parameters. As alluded to above_ , the issue is name collisions. The parameters Concatenate[("n", int), P] are only valid when P itself does not already have a parameter named n\ .

Calling problem(2) works fine, but calling problem(n=2) leads to a TypeError: problem() got multiple values for argument 'n' from the call to added inside of innocent_wrapper\ .

This kind of situation could be avoided, and this kind of decorator could be typed if we could reify the constraint that a set of parameters **not** contain a certain name, with something like:

The call to add_n inside of innocent_wrapper could then be rejected since the callable was not guaranteed not to already have a parameter named n\ .

However, enforcing these constraints would require enough additional implementation work that we judged this extension to be out of scope of this PEP. Fortunately the design of ParamSpec\ s are such that we can return to this idea later if there is sufficient demand.

Naming this a ParameterSpecification

We decided that ParameterSpecification was a little too long-winded for use here, and that this style of abbreviated name made it look more like TypeVar.

Naming this an ArgSpec

We think that calling this a ParamSpec is more correct than referring to it as an ArgSpec, since callables have parameters, which are distinct from the arguments which are passed to them in a given call site. A given binding for a ParamSpec is a set of function parameters, not a call-site’s arguments.

Acknowledgements

Thanks to all of the members of the Pyre team for their comments on early drafts of this PEP, and for their help with the reference implementation.

Thanks are also due to the whole Python typing community for their early feedback on this idea at a Python typing meetup, leading directly to the much more compact .args\ /\ .kwargs syntax.

Copyright

This document is placed in the public domain or under the CC0-1.0-Universal license, whichever is more permissive.